InfiniteWeb: Scalable Web Environment Synthesis for GUI Agent Training

Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, Yan Lu

Paper version: arXiv 2601.04126v3 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *training such agents remains challenging due to the scarcity of suitable environments.*

Benchmarks hold only tens-to-hundreds of hand-built apps, and task or trajectory synthesizers reuse those same environments, so training scale never grows.

Why naive generation fails

1. **Consistency.** Independent pages disagree on interfaces.
2. **Correctness.** Code bugs give wrong RL rewards.
3. **Diversity.** Repetitive tasks and styles.

Evaluation setup

OSWorld-Verified · Online-Mind2Web · MobileWorld · generator GPT-5 · GRPO · 600 tasks, 3 runs

- **All online.** OSWorld at a 15-step cap; no offline accuracy anywhere.
- **Held out:** all eval benchmarks are real; synthetic sites are training-only.

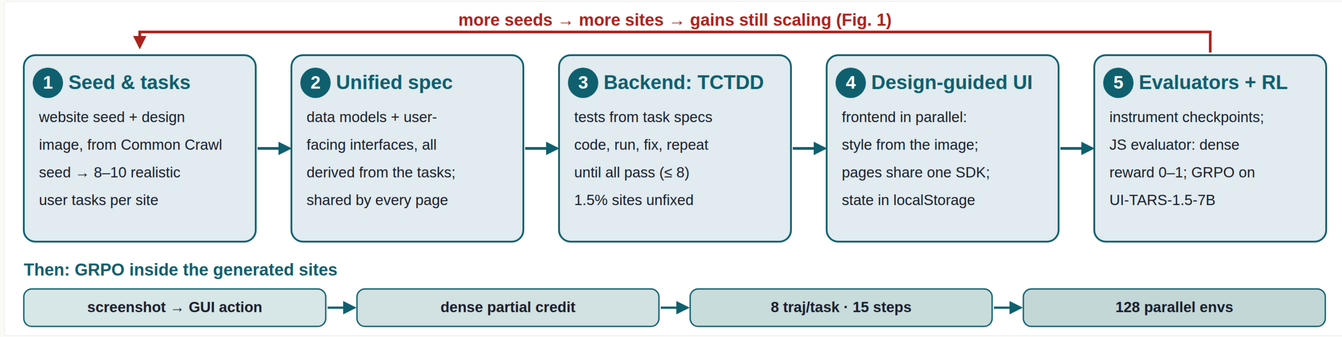
Our read (CUA Radar)

65	★★	transfer
score of 100	band	evidence level

- **Reports:** online eval on 3 real benchmarks; held-out envs; 3-run variance; cost; ablation.
- **Flags:** synthetic-only training mix (AP2); evaluator unvalidated; code announced; baselines partial.
- Sub-pattern K13: checker-first task synthesis. Authors' numbers.

Method: one spec yields the site, the tasks and a dense evaluator

- **Sees / acts:** the trainee is UI-TARS-1.5-7B — screenshot in, low-level GUI actions out; evaluated on web, desktop and mobile.
- **Spec-first:** website seed + Common Crawl design image → 8–10 tasks → shared data models and interfaces → every page.
- **Signal:** TCTDD tests repair the backend; weighted localStorage checkpoints give GRPO a dense 0–1 reward.



Our redraw (not the paper's figure): the four-stage generation pipeline and the GRPO loop it feeds.

$$r = \sum_{c \in C} w_c \, 1[\mathrm{pass}(c)]$$

dense reward: weighted checkpoints (example weights 0.35 / 0.30 / 0.35)

Key results: online success (%)

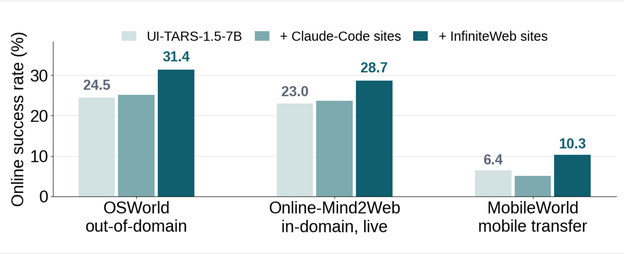
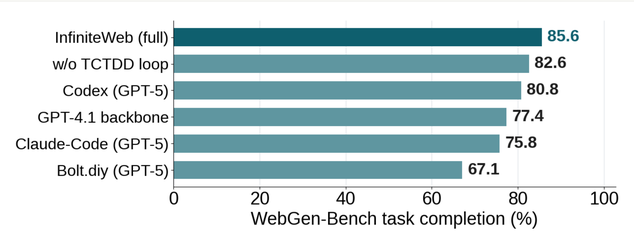


Table 9, redrawn: same UI-TARS-1.5-7B + GRPO, 600-task budget each; mean of 3 runs.

+6.9 OSWorld · +5.7 Online-Mind2Web · +3.9 MobileWorld

SO WHAT → *Std over 3 runs where reported: OSWorld 24.5±1.2 → 31.4±1.0; Online-Mind2Web 23.0±0.9 → 28.7±1.2.*

Ablation: WebGen-Bench (%)



Tables 5–6, redrawn: task completion, mean of 3 runs; all rows use GPT-5 unless stated.

SO WHAT → *The TCTDD loop and the GPT-5 backbone each contribute; even ablated, the pipeline stays above Claude-Code (77.4 vs 75.8). Backbone swap costs 8.2 points; dense reward reaches 767 vs 174 tasks.*

Headline Numbers

31.4%
OSWorld after training
base UI-TARS-1.5-7B: 24.5%

85.6% WebGen-Bench

4.4× RL-usable tasks

$1.93 cost per website

The paper's own controls

CONTROL	INFINITEWEB	BASELINE
Gen quality	85.6 ± 1.2	Codex 80.8
Same GRPO, CC sites	+6.9 OSWorld	+0.7 OSWorld
TCTDD off / GPT-4.1	82.6 / 77.4	full 85.6
Dense vs binary	767 tasks	174 tasks

Takeaway

Checker-first synthesis: one spec yields the site, the tasks and a dense evaluator; GRPO on 600 synthetic tasks lifts a 7B GUI agent on three real online benchmarks.

85.6 WebGen-Bench · $1.93 / website · 4.4× RL-usable tasks · 95/100 evaluators pass · 1.5% sites unfixed

Scan to read the full paper

arxiv.org/abs/2601.04126v3